Optimizing token usage in an AI agent

A. Techniques that reduce token usage

1. **Retrieval tuning** (reduces input). Improve how the agent searches and selects passages before sending them to the model, using top-k, chunk size (the size of each piece of a document) and a similarity threshold (drops passages that match the question too weakly). *Benefit:* fewer tokens, lower cost, faster answers. *Risk:* leaving out important information, and extra processing time.

2. **History management** (reduces input). Send the last 5 messages plus a short summary of the earlier conversation. *Benefit:* fewer previous messages sent, lower cost. *Risk:* the summary can drop important details, so the AI forgets or misunderstands earlier context, and the summary needs an extra model call.

3. **Prompt compaction and prompt caching**. Compaction makes the instructions and context shorter while keeping the meaning (reduces input; risk: removing too much and the AI misunderstands). Caching reuses the repeated start of a prompt so the provider does not process it again. It lowers cost and response time but not the number of tokens, only helps when prompts repeat, and depends on provider support. It is different from disk caching, which returns saved answers with no API call.

4. **Output control** (reduces output). Ask for concise answers and set a maximum token limit (`max_tokens`). Example: 200 tokens become 80. *Risk:* answers get cut off or leave out facts. Reasoning models count hidden thinking as output tokens, so a concise instruction does not remove those.

5. **Model routing** (neither input nor output directly). Use a smaller model for simple questions and a larger one for hard questions. It mainly saves money, not tokens. *Risk:* a hard question sent to the small model gets a worse answer, and choosing the model adds a little cost and time.

B. The two techniques I would test first

Retrieval tuning. *Why:* retrieved passages are the biggest part of the input, so this is where most tokens can be saved. *How:* compare the baseline agent with the optimized one on the same questions, change one setting at a time (top-k, chunk size, threshold), and choose settings on dev questions before comparing on separate test questions. *Metrics:* input tokens per request (from the API usage numbers), retrieval hit rate, answer correctness, latency. *Quality check:* the correct passage is still retrieved, the answer contains the key facts, and a second model (a judge) marks it correct.

Output control. *Why:* we control how much text the AI writes, and it is easy to test. *How:* a concise-answer instruction plus a max_tokens limit, compared with the baseline on the same questions. *Metrics:* output tokens per answer, answers cut off by the limit, latency. *Quality check:* shorter answers could miss facts or be cut off, so I use the same key-fact check and judge, and count the answers that stopped because of the limit.

Decision rule (written before running): a change is accepted when tokens drop by at least 30% and at most 1 of the 12 test questions gets worse.

C. Experiment and proof of concept

Setup. A small question answering agent over 7 pages of the official Python 3.14 documentation (newer than the model's training data). *Baseline:* 300-word chunks, top-8 passages, normal instructions. *A:* retrieval tuning (100-word chunks, top-4, chosen on 8 dev questions). *B:* output control (at most two sentences, max_tokens 300). *A+B:* both. Retrieval is TF-IDF with cosine similarity, chosen over embeddings to stay simple and fast. Answer model: openai/gpt-oss-20b on Groq. Judge: qwen/qwen3.8-27b, a different model family. 20 questions: 8 dev (to choose settings) and 12 test (for results), including 3 that need two passages and 3 that cannot be answered from the documents.

Measured. Prompt, completion and total tokens from the API usage numbers, retrieval hit rate (is the evidence in the retrieved text), correctness (key-fact rule check and the judge), answers cut off, latency.

Testing and validation. 22 automatic tests (chunking, retrieval, metrics, question file, API wrapper). A test checks that every evidence phrase of every question exists in the stored documentation. A no-context check showed the model already knew 2 questions without the documents and invents answers to unanswerable ones. Answers are cached on disk, so a run can be repeated without new API calls.

variant	avg total tokens	vs baseline	avg output tokens	hit rate	correct (judge)
baseline	3380	0%	84	10/10	12/12
A retrieval	720	-79%	77	9/10	11/12
B output	3368	-0.4%	59	10/10	12/12
A+B	714	-79%	59	9/10	11/12

12 test questions, one run each. No answer was cut off. Latency went from 0.6 s to 0.3 s with A.

Results and lessons learned.

- About 97% of the tokens are input, so retrieval tuning matters far more than output control. B cut output tokens by about 30%, but total tokens barely changed. By my rule A and A+B pass (at the limit) and B fails on total tokens. B may still matter for cost because output tokens are usually priced higher, but I did not measure cost.
- The one answer A lost needed two passages. One was not retrieved, and the model gave a wrong answer instead of saying it did not know.
- The similarity threshold had no effect on this data.
- My first rule check was too strict ("8 %" vs "8%", different ways of saying "not in the documents"). The judge was right each time, so I fixed the rule check, and it now agrees with the judge on all 48 answers.
- Reasoning tokens count as output, so a concise instruction does not remove them.

Limits. Only 12 test questions and one run each, so a one question difference is weak evidence. A was tuned on only 7 answerable dev questions. TF-IDF instead of embeddings, one answer model, and the judge is a model too. The documentation was fetched with an AI-assisted tool, so small differences from the live pages are possible.

AI tools used

Planning: Claude helped plan the exercise: which techniques to describe, which two to test the experiment design and the schedule.
Explanations: Claude and ChatGPT explained concepts (tokens, caching, chunking, top-k, reasoning tokens, LLM judges) so that I can explain them myself, Since i wasn't familiar with this.
Code: I wrote the codes ` src/` and in some of the ` tests/`(API call, token counts, latency). Claude reviewed it, helped fix it, and gave better solution or approaches.
Data: Claude drafted the 20 questions in `data/questions.jsonl` from those pages i found. A test checks that every evidence phrase exists in the pages.
Report: I wrote the notes for Sections A and B in my own words. Claude fixed grammar and added some sentences, which I reviewed and kept or changed. Details are in AI_USAGE.md. Code and raw results: github.com/Eddy-Prime/AI-Agent-Exercise2Nov2026

D. References

- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- Liu et al. (2023). Lost in the Middle: How Language Models Use Long Contexts.
- Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.
- Groq API documentation (usage fields, rate limits, reasoning_effort), console.groq.com/docs.
- Python 3.14 documentation, docs.python.org/3.14 (the 7 pages stored in data/docs).